35、解决kernel 5.2.y wifi驱动问题

全志V3S嵌入式驱动开发（解决kernel 5.2.y wifi驱动问题）

原创

嵌入式-老费 已于 2023-08-29 15:25:32 修改

阅读量714 收藏

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 本文详细记录了解决全志V3S芯片在Linux内核5.2.y版本下ESP8089 WiFi驱动的问题。从重新编译驱动代码，拷贝wpa_supplicant和驱动到U盘，配置wpa_supplicant.conf，到分析日志，尝试多种解决方案，最终发现内核无线通信配置差异导致的问题，并成功解决。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

wifi模块，之前测试的时候，开发板上用的是esp 8089，当时内核时4.14.y，测试结果也是通过的。印象不是太深的朋友可以回顾一下，链接地址在这，

```
https://blog.csdn.net/feixiaoxing/article/details/131178420?spm=1001.2014.3001.5501
```

那个时候测试，我们就说过，wifi驱动可能是除了镜像制作之外，最复杂的一个驱动工作。因为它除了电路分析、设备树、驱动编译之外，还得自己编译wpa_supplicant等工具。这次内核升级成5.2.y，我们也需要回过头来看下，wifi驱动在新内核是不是还能编译成功，是不是还能运行得起来。

1、重新编译esp 8089驱动代码

因为之前下载了esp 8089的驱动代码，我们也升级了内核，所以第一步就是对驱动进行重新编译。说是重新编译，本质来说，就是将4.14.y 内核目录替换成新内核 5.2.y 的路径就可以了，

```
ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- make -C ~/Desktop/linux-zero-5.2.y M=${PWD}  modules
```

2、拷贝其他驱动和wpa_supplicant软件到u盘，准备测试

在解决5.2.y内核版本网卡驱动的时候，发现很多驱动都以ko的形式存在了。我们的解决办法就是把对应的驱动改成y，这样比较简单。当然，这么做是有后果的。后果就是zImage文件的大小变了很多。这不是说，zImage就不能增加，关键是zImage发生了改变，那么uboot、rootfs的布局就要发生改变，一连串的修改就要去完成，所以这有点麻烦。最后，我们想到的办法，就是用insmod安装ko文件好了，虽然麻烦点，但是好歹也能解决驱动问题。

```
feixiaoxing@feixiaoxing-VirtualBox:~/Desktop/linux-zero-5.2.y$ find . -name "*.ko"
./drivers/video/backlight/lcd.ko
./drivers/input/mousedev.ko
./drivers/staging/rtl8723bs/r8723bs.ko
./drivers/usb/class/cdc-wdm.ko
./drivers/usb/serial/usbserial.ko
./drivers/usb/serial/cp210x.ko
./drivers/usb/serial/usb-serial-simple.ko
./drivers/usb/common/usb-otg-fsm.ko
./lib/crypto/libarc4.ko
./net/ipv4/tcp_westwood.ko
./net/ipv4/tcp_bic.ko
./net/ipv4/udp_diag.ko
./net/ipv4/tcp_htcp.ko
./net/wireless/cfg80211.ko
./net/mac80211/mac80211.ko
```

通过分析上面这个搜索结果，大体也能看得出来。除了esp 8089的驱动外，cfg80211.ko、mac80211.ko、libarc4.ko这三个动态库文件也是用的上的。至于tcp_westwood.ko、tcp_bic.ko、udp_diag.ko、tcp_htcp.ko这四个文件是不是用得上，我们到时候再看。从名字上说，他们都是和网络相关的，但是从内容上说，前面三个更加紧密一点。

当然除了驱动之外，wpa_supplicant也是很重要的。它最大的作用就是帮助嵌入式设备连接上wifi ap。只有连接上了ap，我们的wlan0才能配置ip、进行后续的工作。

3、准备wpa_supplicant.conf和拷贝libnl的动态库文件

wpa_supplicant.conf主要是记录wifi ssid和密码，准备一份即可，

```
ctrl_interface=/var/run/wpa_supplicant
network={
    ssid="xiaoyan_home"
    psk="88888888"
}
```

而libnl的动态库，则是运行wpa_supplicant需要的，所以我们有必要也把libnl的动态库拷贝到u盘上面。当然，执行的时候需要把路径带上就行。此外，如果因为链接不太

好copy，可以通过cp命令直接来解决，

```
cp libnl-3.so.200.18.0 libnl-3.so.200
cp libnl-idiag-3.so.200.18.0 libnl-idiag-3.so.200
cp libnl-cli-3.so.200.18.0 libnl-cli-3.so.200
cp libnl-nf-3.so.200.18.0 libnl-nf-3.so.200
cp libnl-genl-3.so.200.18.0 libnl-genl-3.so.200
cp libnl-route-3.so.200.18.0 libnl-route-3.so.200
```

最后，如wpa_supplicant.conf所示，到时候还需要创建一个目录，它的名称就是/var/run/wpa_supplicant，这个需要注意下。

4、开始测试

因为测试的内容比较多，所以我们可以写成脚本一步一步来。首先，第一步肯定是插好u盘，安装好基础驱动，

```
mount /dev/sda1 /mnt
cd /mnt/v3s
insmod libarc4.ko
insmod cfg80211.ko
insmod mac80211.ko
```

等第一步没问题，下面就是安装esp 8089驱动，

```
insmod esp8089.ko esp_reset_gpio=43
```

驱动安装好了，接着就是为执行wpa_supplicant做准备，

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/mnt/v3s/lib
ifconfig wlan0 up
mkdir -p /var/run/wpa_supplicant
```

有了上面这些铺垫，终于到了最重要的一步，那就是利用wpa_supplicant看一下，wlan0是不是可以连接上ap节点。连接上，那么问题不大，否则就有点麻烦了，

```
./wpa_supplicant -D wext -c /mnt/v3s/wpa_supplicant.conf -i wlan0 &
```

让我们没想到是，这前面的步骤都很顺利，就是这最后一步出了麻烦，

```
# ./wpa_supplicant -D wext -c /mnt/v3s/wpa_supplicant.conf -i wlan0 &
# Successfully initialized wpa_supplicant
rfkill: Cannot open RFKILL control device
ioctl[SIOCSIWMODE]: Operation not supported
ioctl[SIOCGIWRANGE]: Operation not supported
ioctl[SIOCGIWMODE]: Operation not supported
ioctl[SIOCSIWAP]: Operation not supported
ioctl[SIOCSIWESSID]: Operation not supported
ioctl[SIOCSIWENCODEEXT]: Operation not supported
ioctl[SIOCSIWENCODE]: Operation not supported
ioctl[SIOCSIWENCODEEXT]: Operation not supported
ioctl[SIOCSIWENCODE]: Operation not supported
ioctl[SIOCSIWENCODEEXT]: Operation not supported
ioctl[SIOCSIWENCODE]: Operation not supported
ioctl[SIOCSIWENCODEEXT]: Operation not supported
ioctl[SIOCSIWENCODE]: Operation not supported
ioctl[SIOCSIWENCODEEXT]: Operation not supported
ioctl[SIOCSIWENCODE]: Operation not supported
ioctl[SIOCSIWENCODEEXT]: Operation not supported
ioctl[SIOCSIWENCODE]: Operation not supported
ioctl[SIOCSIWSCAN]: Operation not supported
wlan0: CTRL-EVENT-SCAN-FAILED ret=-1 retry=1
ioctl[SIOCSIWSCAN]: Operation not supported
```

5、尝试解决问题

5.1 尝试配置RFKILL

解决问题的第一步，肯定是观察现象，分析日志。从日志当中，我们看到了这么一句，

```
rfkill: Cannot open RFKILL control device
```

后来查资料说，是menuconfig里面需要配置下RFKILL。结果是我们重新配置、编译了，结果没有改善。

5.2 查看dmesg信息

内核信息一般用dmesg进行查看、分析的，不过很可惜，没有特别有用的信息。

5.3 将其他udp/tcp的ko文件也加进来

走这一步有点病急乱投医的味道，安装驱动倒是没有什么问题，结果没有改善。

5.4 将wpa_supplicant和lib文件拷贝到root目录

因为打印日志上面，有关于权限的打印，所以我们的一个直觉就是，是不是执行文件的权限设置错了。所以，就把wpa_supplicant和依赖的lib文件都拷贝到root目录下来执行，

```
ioctl[SIOCSIWMODE]: Operation not supported
ioctl[SIOCGIWRANGE]: Operation not supported
```

思路没啥问题，可是没有解决问题。

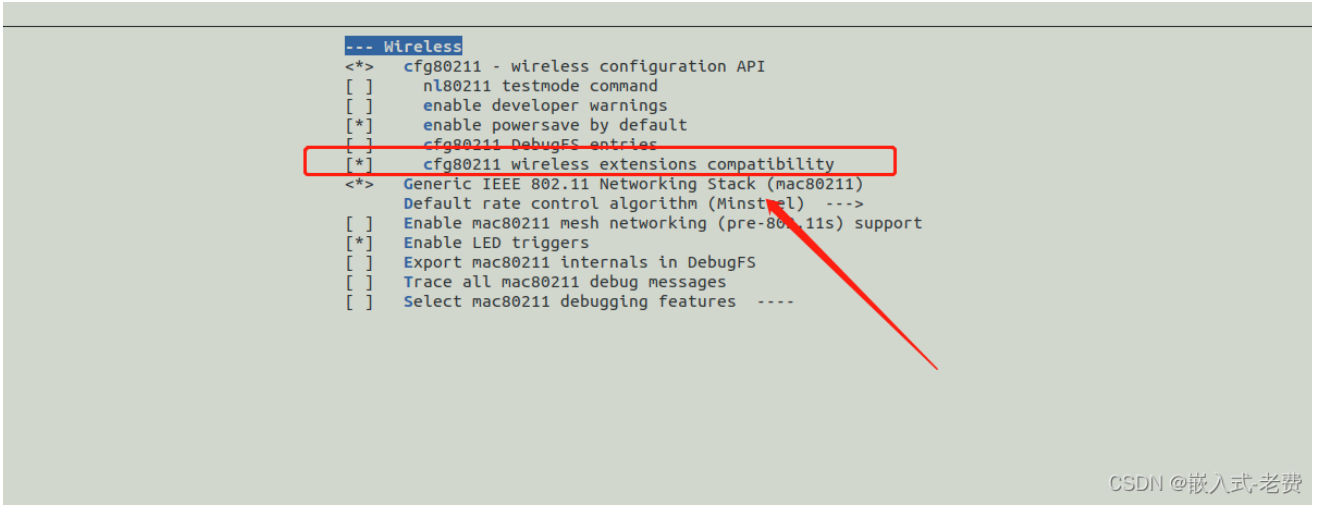
5.5 重新编译wpa_supplicant

之前wpa_supplicant和kernel 4.14.y搭配没问题的，但是换成kerenel 5.2.y就不行了。那是不是wpa_supplicant重新编译一下就好了，毕竟升级了内核，而且我们还替换了编译器。wpa_supplicant之前依赖的文件代码都还在，本身编译过程不复杂，重新做一次就好了。只是结果很失望，没有解决问题。

6、灵光一闪

有一个很奇怪的现象，那就是wpa_supplicant和4.14.y kernel搭配就没有问题，和现在的kernel搭配就有问题，而驱动代码都没有变。这说明wpa_supplicant和esp8089出问题的可能性不大，问题还是出在5.2.y kernel的配置上，带着这个疑问，我们比较了一下两者的menuconfig配置差别。没想到还真的有不一样的地方，

在4.14.y kernel上面，关于无线通信的配置，是这么做的，



通过和5.2.y kernel上面比较，这其中cfg wireless extensions compatibility是没有选上的。所以，我们猜测原因很有可能是这个。除此之外，wpa_supplicant上面有一

个wext选项，估计也是wireless extensions的缩写。如果对应的内容没有选上，那么命令当然就不得不到执行了。

所以，我们选上wext，快速生成zImage和ko文件之后，重新烧录了一把，没想到还真的解决了问题，

```
# ./wpa_supplicant -D wext -c /mnt/v3s/wpa_supplicant.conf -i wlan0 &
# Successfully initialized wpa_supplicant
ioctl[SIOCSIWENCODEEXT]: Invalid argument
ioctl[SIOCSIWENCODEEXT]: Invalid argument
wlan0: Trying to associate with 3c:06:a7:b3:e3:3e (SSID='xiaoyan[   77.589525] wlan0: authenticate with 3c:06:a7:b3:e3:3e
_home' freq=2472 MHz)
Failed to add supported operating classes[   77.599718] wlan0: send auth to 3c:06:a7:b3:e3:3e (try 1/3)
 IE
[   77.615046] wlan0: authenticated
[   77.625720] wlan0: associate with 3c:06:a7:b3:e3:3e (try 1/3)
[   77.636004] wlan0: RX AssocResp from 3c:06:a7:b3:e3:3e (capab=0x1411 status=0 aid=1)
[   77.643929] wlan0: associated
wlan0: Associated with 3c:06:a7:b3:e3:3e
wlan0: WPA: Key negotiation completed with 3c:06:a7:b3:e3:3e [PTK=CCMP GTK=CCMP]
wlan0: CTRL-EVENT-CONNECTED - Connection to 3c:06:a7:b3:e3:3e completed [id=0 id_str=]

# ifconfig wlan0 192.168.0.231 netmask 255.255.255.0 broadcast 192.168.0.1
# ping 192.168.0.100
PING 192.168.0.100 (192.168.0.100): 56 data bytes
64 bytes from 192.168.0.100: seq=0 ttl=128 time=36.303 ms
64 bytes from 192.168.0.100: seq=1 ttl=128 time=8.744 ms
64 bytes from 192.168.0.100: seq=2 ttl=128 time=61.449 ms
64 bytes from 192.168.0.100: seq=3 ttl=128 time=85.053 ms
64 bytes from 192.168.0.100: seq=4 ttl=128 time=193.609 ms
64 bytes from 192.168.0.100: seq=5 ttl=128 time=61.826 ms
64 bytes from 192.168.0.100: seq=6 ttl=128 time=53.610 ms
^C
```

CSDN @嵌入式-老费

回过头来看，整个过程的解决虽然花费了一天多的时间，但是好歹学了一点东西。此外，对于曾经成功的配置，一定要注意保存，不要轻易地丢弃、或者处理掉。